Software Project Management (SPM)

Software Project Management (SPM) is the process of **planning, organizing, leading, and controlling** a software project to achieve specific goals within **time, cost, and quality constraints**.

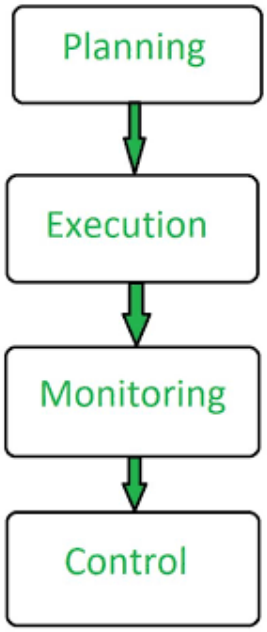
It ensures that:

- The project is delivered **on time**
- The **budget is maintained**
- The software meets **user requirements and quality standards**

Objectives of Software Project Management

- Deliver the **right product with required quality**
- Minimize **risks during development**
- Ensure efficient use of **resources (people, tools, time)**
- Handle **change management** effectively
- Achieve **customer satisfaction**

Process in SPM



<u>Key Activities in Software Project Management</u>

1. Project Planning

- Defines **scope, objectives, and deliverables**
- Estimates **time, cost, and resources**
- Selects suitable development models:
    - Agile
    - Waterfall
- Forms the foundation for the entire project.

2. Project Monitoring and Control

- Tracks **progress, quality, and cost**
- Compares actual performance with planned schedule
- Identifies **deviations and risks**
- Take **corrective actions** to keep the project on track.

---

3. Team Management

- Assigns tasks and defines **roles and responsibilities**
- Ensures **effective communication and coordination**
- Motivates team members for better productivity
- Handles conflicts within the team.

---

4. Risk Management

- Identifies potential risks such as:
    - Schedule delays
    - Budget overruns
    - Scope creep
- Analyzes risk impact and probability

- Develops **mitigation strategies**
- Continuously monitors risks throughout the project.

---

5. Quality Management

- Defines **quality standards and guidelines**
- Conducts **reviews, testing, and audits**
- Ensures the final product meets **customer expectations**.

6. Customer and Stakeholder Management

- Collects and analyzes **requirements and feedback**
- Maintains regular communication with stakeholders
- Ensures alignment between **project output and customer needs**.

Challenges in Software Project Management

- Frequently changing requirements
- Errors in time and cost estimation
- Poor team coordination and communication
- Technological limitations
- Conflicts among stakeholders.

Tools Used in Software Project Management

Project Management Tools

- Microsoft Project
- Trello
- Jira

Communication Tools

- Slack
- Zoom

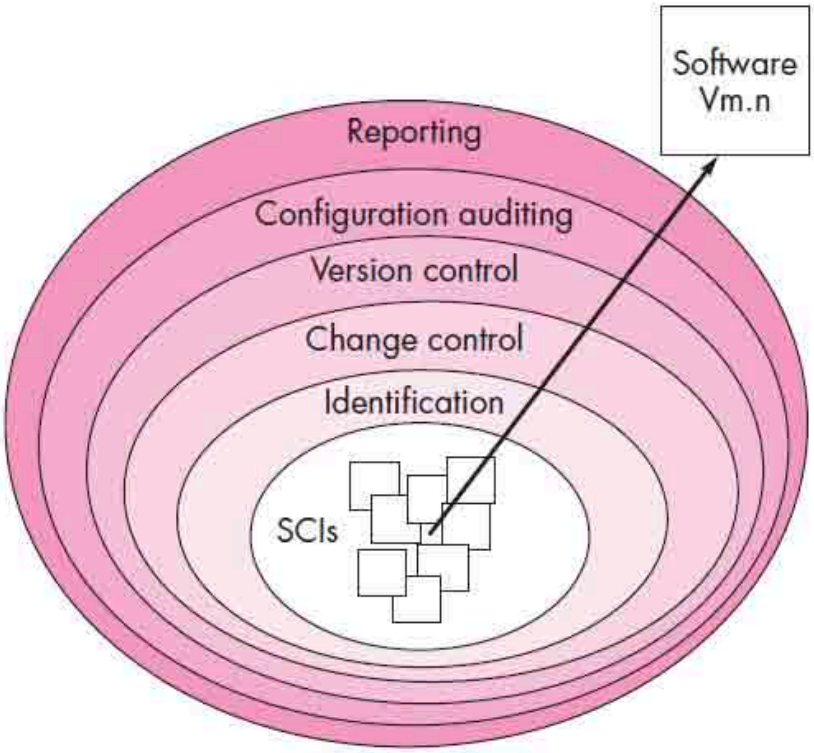
Time Tracking Tools

- Toggl
- Clockify

Software Configuration Management (SCM)

Software Configuration Management

Software Configuration Management (SCM) is the process of **managing, tracking, and controlling changes** in software during its development life cycle. It helps to make sure that **all changes in software are recorded, controlled, and organized properly**.



Need for SCM

SCM is important because software projects involve **many changes and multiple developers**.

Reasons:

- Avoid confusion between different versions
- Track changes made in code
- Manage team work efficiently
- Prevent errors due to uncontrolled changes
- Maintain consistency in the system

Objectives of SCM

- Control changes in software
- Maintain multiple versions of software
- Ensure product integrity and consistency
- Improve team coordination
- Provide traceability of changes

Key Activities of SCM

1. Configuration Identification

This step identifies all the important items in the software.

Example:

- Source code
- Design documents
- Test cases

These are called **Configuration Items (CI)**.

---

2. Version Control

Version control manages different versions of software.

Example:

- Version 1.0
- Version 2.0
- Version 2.1

It helps developers:

- Track changes
- Go back to previous versions
- Work on the same project without conflict

---

3. Change Control

Change control ensures that all changes are **properly reviewed and approved**.

Process:

1. Request for change
2. Analyze impact
3. Approve or reject
4. Implement change

Prevents unwanted or risky changes.

---

4. Configuration Status Accounting

This activity records and tracks the status of all configuration items.

Example:

- Which version is current?
- What changes are made?
- Who made the change?

Helps in project tracking.

---

5. Configuration Auditing

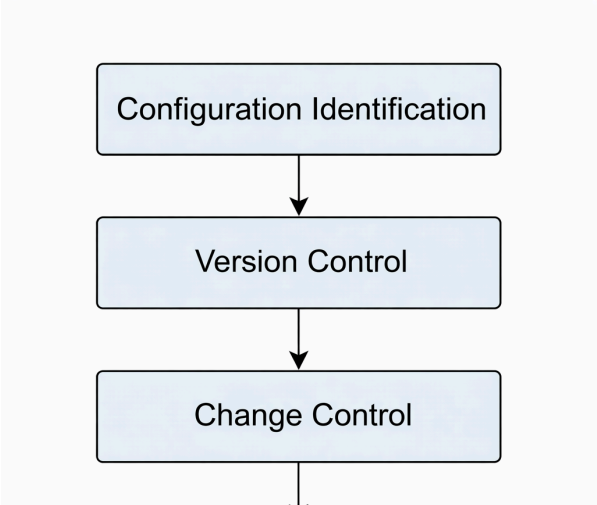
Auditing checks whether the software is correct and meets requirements.

Types:

- Functional Audit → checks functionality
- Physical Audit → checks documents and files

Ensures quality and correctness.

<u>SCM Process Flow</u>



SCM Tools

Some tools used in SCM:

- Git
- SVN (Subversion)
- Mercurial

These tools help in:

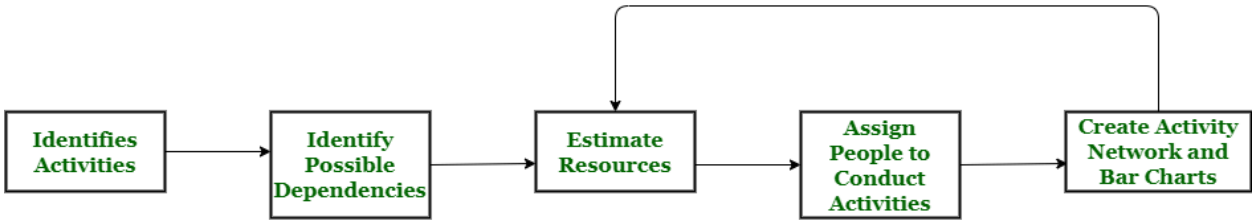
- version control
- collaboration
- tracking changes

Advantages of SCM

- Better control over software changes
- Easy team collaboration
- Reduced errors and conflicts
- Easy maintenance and updates
- Improved software quality

Project Scheduling

Project Scheduling is the process of **planning, organizing, and managing tasks, timelines, and resources** to ensure that a software project is completed **on time and within budget**.



Project Scheduling Process

Objectives / Goals of Project Scheduling

- Efficient allocation of resources
- Identification of task dependencies
- Determination of the critical path
- Monitoring progress and delays
- Ensuring the project stays on track

Key Elements of Project Scheduling

1. Work Breakdown Structure (WBS)

- Breaks the project into smaller, manageable components
- Helps in assigning responsibilities
- Makes tracking and control easier

2. Task Estimation

- Determines the effort and duration required for each task
- Techniques used:
    - Expert judgment
    - Analogy-based estimation
    - PERT (Program Evaluation and Review Technique)

3. Task Dependencies

- Defines the relationship between tasks
- Types:
    - Finish-to-Start (FS)
    - Start-to-Start (SS)
    - Finish-to-Finish (FF)

4. Milestones

- Important checkpoints in the project timeline
- Examples:
    - Completion of design
    - Prototype development
    - Final delivery

5. Critical Path Method (CPM)

- Identifies the **longest sequence of dependent tasks**
- Determines the minimum project completion time
- Delay in critical tasks leads to delay in the entire project

6. Gantt Charts

- Graphical representation of the project schedule

- Displays:
    - Task durations
    - Start and end dates
    - Dependencies
    - Progress tracking

Benefits of Project Scheduling

- Improves project visibility and control
- Ensures better resource utilization
- Enhances time management
- Helps in risk identification and proactive planning

Tools for Project Scheduling

- MS Project
- Trello
- Asana
- Jira
- Wrike
- Gantt Chart tools
- PERT Chart tools

DevOps in Software Engineering

Introduction

DevOps is a combination of two words: **Development** and **Operations**. It is a software engineering practice where developers and IT operations teams work together throughout the software development life cycle. The main aim of DevOps is to deliver software faster, with better quality, and fewer errors.

Unlike traditional siloed methodologies, it overcomes communication barriers and delays through:

- **Enhanced teamwork**
- **Automation of processes**
- **Continuous delivery frameworks**

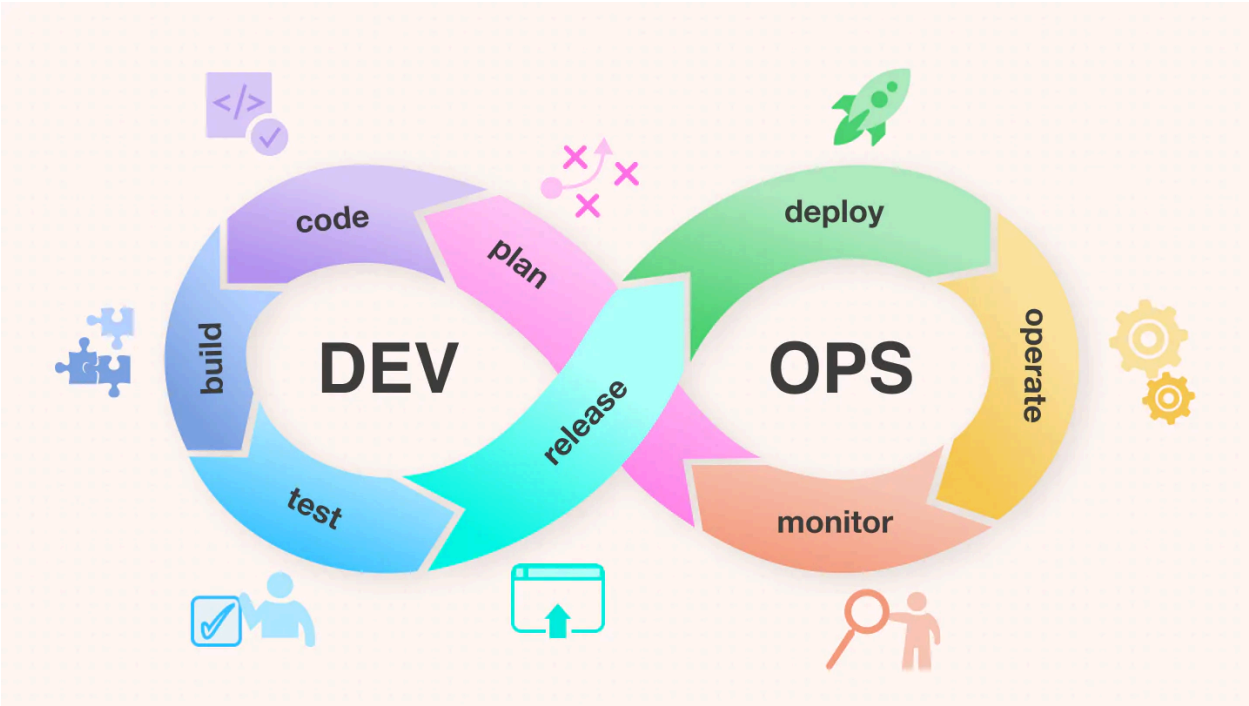
Definition

DevOps is a comprehensive **culture, process, and toolset** designed to facilitate efficient building, testing, and maintenance of software systems.

Key Principles of DevOps

- **Collaboration**: Shared responsibility between dev and ops.
- **Automation**: Streamlining testing and deployment.
- **Continuous Integration (CI)**: Regular merging of code changes.
- **Continuous Delivery (CD)**: Rapid and reliable software releases.
- **Monitoring**: Real-time tracking of system health.
- **Feedback**: Fast loops to drive product evolution.

DevOps Life Cycle



1. **Plan** – Requirements and project goals are decided before development starts. The team discusses features, cost, and time needed.
   Example: Planning a login page and user dashboard for a website.
2. **Develop** – Developers write the source code based on the planned requirements. New features are created in this stage.
   Example: Coding the login page using Java or Python.
3. **Build** – The written code is compiled and converted into an application. It combines all files into usable software.
   Example: Creating an APK file for an Android app.
4. **Test** – Software is checked to find bugs, errors, and performance issues. This ensures quality.
   Example: Testing whether login works with correct and wrong passwords.
5. **Release** – The tested software is prepared for deployment. Final approval is given for launch.
   Example: Version 1.0 of the app is made ready for users.
6. **Deploy** – Software is installed on servers or delivered to users. Users can now access the application.
   Example: Uploading the website to a cloud server.

7. **Operate** – The software runs in a real environment and is maintained regularly. Support is given to users.
   Example: Restarting server if website becomes slow.
8. **Monitor** – System performance is continuously observed to detect issues. Feedback helps future improvement.
   Example: Checking server traffic and fixing crashes quickly.

Popular DevOps Tools

- **Git** – Version control platform
- **Jenkins** – CI/CD automation
- **Docker** – Application containerization
- **Kubernetes** – Orchestration engine
- **Ansible** – Infrastructure automation
- **Nagios** – Monitoring and alerting

Advantages of DevOps

- Accelerated lifecycle from development to delivery
- Optimized team synergy and communication
- Significant reduction in defect density
- Rapid disaster recovery capabilities
- Enhanced organizational productivity
- Superior user and customer experience
- Minimized operational expenditure

Challenges of DevOps

- Necessity for organizational cultural shifts
- Demand for highly specialized skill sets
- Substantial initial infrastructure investment
- Complexities in integrated security management
- Mandatory requirements for constant monitoring.

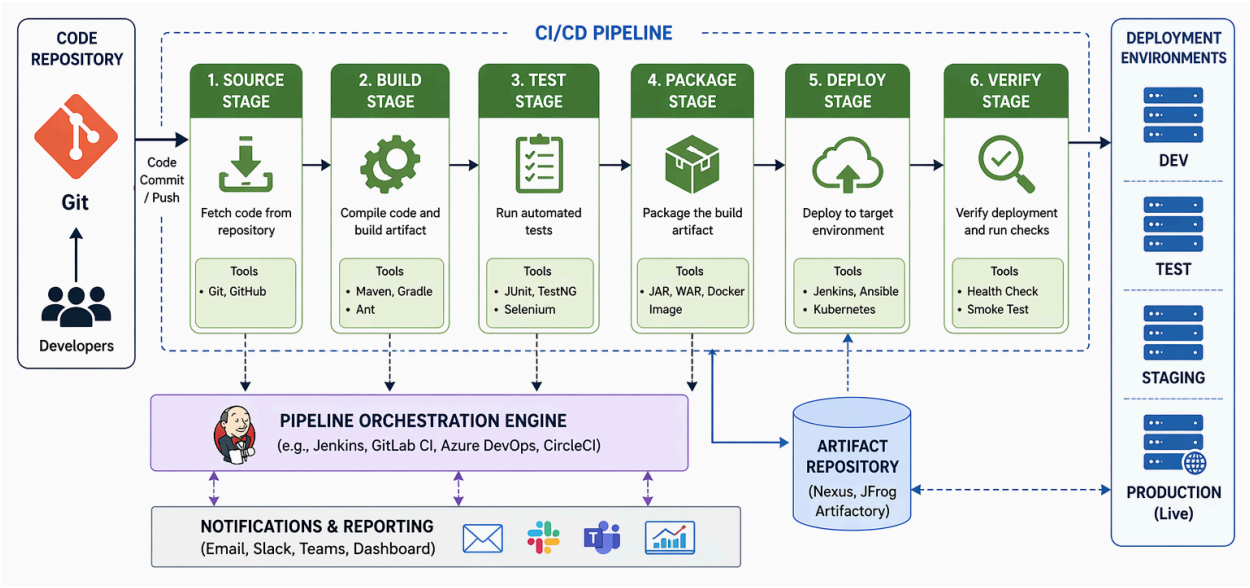
Deployment Pipeline Architecture

Introduction

Deployment Pipeline Architecture is a structured process used in **DevOps** to automate software delivery. It helps move the software from source code stage to production stage through different steps like build, test, and deployment. It ensures fast delivery, better quality, and fewer errors.

A deployment pipeline is also called a **CI/CD Pipeline** because it uses Continuous Integration and Continuous Delivery practices.

Architecture of Deployment Pipeline



The main stages of deployment pipeline are:

1. Source Code Management

Developers write code and store it in a version control system like Git or GitHub. Every code commit triggers the pipeline.

Example: Developer uploads login module code to GitHub.

2. Build Stage

The source code is compiled and converted into executable files. Dependencies and packages are created.

Tools: Maven, Gradle, Ant

Example: Java code is compiled into a JAR file.

3. Automated Testing Stage

Software is tested automatically using unit testing, integration testing, and performance testing. Errors are detected early.

Tools: JUnit, Selenium, TestNG

Example: Checking whether the login page works correctly.

4. Packaging Stage

After successful testing, software is packed into deployable format.

Example: Creating WAR file, Docker image, ZIP package.

5. Deployment Stage

The packaged software is deployed to testing, staging, or production servers automatically.

Tools: Jenkins, Docker, Kubernetes, Ansible

Example: Uploading application to cloud server.

6. Verification Stage

After deployment, health checks and smoke testing are performed to verify successful installation.

Example: Checking website opens without error.

7. Monitoring and Feedback

Application performance is monitored continuously. Feedback is sent to developers for improvement.

Example: CPU usage, server logs, crash reports.

Common Tools Used

Stage	Tools
Source Control	Git, GitHub
Build	Maven, Gradle
Test	Selenium, JUnit
Deploy	Jenkins, Docker
Monitor	Nagios, Prometheus

Benefits of Deployment Pipeline

1. Faster software release
2. Better software quality
3. Reduced human errors
4. Repeatable process
5. Easy rollback if failure occurs
6. Continuous improvement
7. Customer satisfaction